



MSc. Artificial Intelligence

Continuous Assessment Number 2: Data Acquisition and Transformation

Student Name: Nathan Lucio de Carvalho Rocha

Student ID: 20082900

Module: Programming for Data Analysis

Lecturer: Paul Laird

Submission Date: April 2026

1 Introduction

This project was developed as part of the Continuous Assessment for the module *Programming for Data Analysis (B9AI001)*.

The main goal of the project is to build a data acquisition and preprocessing pipeline to collect job posts from the Gupy platform (<https://portal.gupy.io/>) and transforming this data into a structured format suitable for analysis.

The system is implemented as a Flask-based application that handles data collection, processing, and storage. SQLAlchemy is used as the Object-Relational Mapping (ORM) tool to interact with the database, while Alembic is used for database version control and migrations. Swagger is integrated to automatically generate API documentation, improving usability and transparency of the system.

For data storage, a PostgreSQL database is used to persist the processed information in a structured and normalized format.

The application can be deployed either as a standalone Flask service or using Docker Compose, which allows the application and the PostgreSQL database to run in separate, containerized environments.

2 Data Source

The data used in this project was collected from the Gupy platform, a recruitment system used in Brazil. By inspecting the browser developer tools, it was possible to identify that job listings are retrieved through an API endpoint rather than being statically loaded on the page.

2.1 API Endpoint

The main endpoint used is:

`https://employability-portal.gupy.io/api/v1/jobs`

This endpoint accepts several query parameters to control the results, such as:

- `jobName`: used to search for specific job titles
- `limit`: number of jobs returned per request
- `offset`: used for pagination

- **sortBy:** field used to sort results
- **sortOrder:** sorting direction

The API returns job postings in JSON format, where each job contains information such as title, company, publication date, location, and a full job description. While some fields are already structured, many important details—such as required skills, responsibilities, and salary—are only available inside the description as free text.

2.2 Target Features for Extraction

The main objective of the transformation stage is to extract structured attributes from unstructured job descriptions. The key features targeted in this project are listed below:

- **Salary:** Salary for the position. When available, salary values are extracted and normalized into numerical ranges.
- **Seniority Level:** Indicates the experience level required for the position. The categories considered include:
 - Senior (*Sênior*)
 - Mid-level (*Pleno*)
 - Junior (*Júnior*, when present)
 - Leadership (*Liderança*)
 - Specialist (*Especialista*)
 - Manager (*Gerente*)
 - Internship (*Estágio*)
 - Consultant (*Consultor*)
- **Contract Type:** Refers to the employment model. The most common types in Brazil are:
 - CLT (formal employment contract)
 - PJ (contractor / legal entity)

If the contract type is not explicitly mentioned in the job description, it is assumed to be CLT, as this is the standard hiring model in the Brazilian market.

- **Hard Skills:** Technical requirements required for the role, such as programming languages, frameworks, tools, or platforms (e.g., Python, Java, SQL).

- **Soft Skills:** Non-technical attributes related to behaviour and interpersonal abilities, such as communication, teamwork, problem-solving, and adaptability.

2.3 Analytical Features

In addition to extracting structured attributes from job descriptions, the system also computes a set of aggregated metrics to provide an overview of the Brazilian technology job market.

The following analytical features were implemented:

- **Average Daily Job Posts:** Number of job postings published per day, used to understand market activity over time.
- **Average Salary:** Estimated average salary based on the extracted salary ranges from job descriptions.
- **Top 10 Technologies:** Most frequently required hard skills, ranked by the number of job postings in which they appear.
- **Top 10 Locations:** Cities or regions with the highest number of job postings.
- **Jobs per Contract Type:** Distribution of jobs across different contract types (e.g., CLT and PJ).
- **Jobs per Seniority Level:** Distribution of job postings by experience level (e.g., Junior, Mid-level, Senior).
- **Technology Trend (Last 30 Days):** Temporal analysis of the demand for specific technologies over the most recent 30-day period.

3 Feature Extraction

Three different approaches were explored to extract structured information from the raw job descriptions:

3.1 Regex-Based Extraction

This was the primary method used in the final implementation. Regular expressions were designed to identify specific patterns in the text, such as salary values, contract types, and known technical skills. This approach is fast, efficient, and easy to implement, making it

suitable for large-scale data processing. However, it is limited to predefined patterns and may fail to capture more complex or implicit information.

3.2 LLM-Based Extraction

A Large Language Model (LLaMA via Ollama) was tested to extract features based on semantic understanding of the text. While this approach showed good capability in identifying contextual information, it was significantly slower compared to regex-based methods. Due to performance constraints and the need for processing large volumes of data, this approach was not used in the final pipeline.

3.3 NER-Based Extraction

A Named Entity Recognition (NER) approach was also considered for extracting structured entities such as skills and seniority levels. However, the available pre-trained models did not perform well on the domain-specific and Portuguese job data. Achieving reliable results would require additional training and dataset preparation.

4 Database Design and Data Loading

4.1 Schema Design

After extracting the relevant features from the raw job descriptions, the data is stored in a relational database. The database was designed to organize the information in a structured and efficient way, allowing for easy querying and analysis.

The central entity of the schema is the `Job` table, which stores the main attributes of each job posting, such as title, salary, seniority level, contract type, and publication date. This table is linked to other entities that represent additional information.

To avoid data duplication and improve consistency, separate tables were created for:

- `Company`
- `City`
- `State`
- `HardSkill`
- `SoftSkill`

- `ContractType`
- `NiceToHaveSkills`

4.2 Data Loading Process

The loading process is handled using SQLAlchemy, which maps Python objects to database tables. After the data is extracted and transformed, it is converted into objects and inserted into the database.

To prevent duplication before inserting new records, the system checks if related entities (such as companies or skills) already exist; if they do, the existing records are reused; otherwise, new entries are created.

Transactions are used to ensure that the data is inserted correctly. If an error occurs during the process, the transaction is rolled back to avoid inconsistent states in the database, and the error is logged on the database.

5 Testing

5.1 Unit Testing

Unit tests were created to validate the output from the regex extraction.

For example, salary extraction functions were tested against different text formats to verify that numerical values are correctly identified and normalized. Tests were created for contract type detection and seniority classification to ensure correct behaviour across different job descriptions.

6 Ethical Considerations

6.1 Data Collection and Legality

The data used in this project was collected from publicly accessible job postings on the Gupy platform, using only publicly available endpoints and does not require authentication or access to restricted content.

To minimise impact on the platform, requests were limited and timed to avoid excessive load on the server.

6.2 Data Privacy

The dataset consists of job postings and company-related information. No personal data from applicants or users is collected or stored.

6.3 AI Usage Transparency

Artificial Intelligence tools were used as development support during this project. All AI-assisted interactions, including prompts and responses, are documented in a dedicated folder (`ai-assistance`) included with the submission.

AI tools were primarily used for:

- assisting with code implementation,
- debugging and error resolution,
- improving documentation clarity.